

Universidad Autónoma de Occidente (UAO)

Facultad de Ingeniería — Sistemas Operativos

Taller de Kernel — 28 de Septiembre

Estudiante: Sergio Gómez Quiñones

1. Actividad de laboratorio: FreeBSD

1.1 Identificar el sistema

Estos tres comandos permiten identificar el sistema antes de compilar el kernel:

```
cat /etc/os-release
freebsd-version -u -k -r
getconf LONG_BIT
```

1.2 Configurar el kernel

```
cd /usr/src/sys/amd64/conf
mkdir /root/kernels
cp GENERIC /root/kernels/CLASEKERN
ln -s /root/kernels/CLASEKERN
```

1.3 Compilar e instalar

```
cd /usr/src
make buildkernel KERNCONF=CLASEKERN
make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN
nextboot -k CLASEKERN
init 6
```

1.4 Verificación después de reiniciar

```
uname -a
freebsd-version -k -r
sysctl kern.conftxt | head
```

2. Trabajo investigativo: Tipos de Kernel

- Kernel monolítico (Linux): Todos los servicios corren en un solo espacio. Alto rendimiento, pero un fallo en un driver puede afectar el sistema.
- Kernel con diseño modular (FreeBSD): Carga y descarga módulos (.ko) en caliente sin recompilar.
- Sistema estructurado en capas (THE): Capas numeradas, fácil de depurar pero con costo de rendimiento por atravesar capas.
- Sistema con micrókernel (MINIX 3): El kernel hace lo mínimo (IPC, procesos). Más robusto, menor rendimiento.
- Máquina virtual (IBM VM/370): Un monitor crea copias virtuales del hardware completo para aislar los SO.

2.1 Kernel monolítico: Linux

Origen: Creado por Linus Torvalds en 1991 y publicado bajo licencia GNU GPL. Hoy corre desde teléfonos hasta supercomputadoras.

Organización del kernel: El planificador de procesos, la gestión de memoria, el sistema de archivos (VFS), los drivers y la pila de red están en un único programa que se ejecuta completo en modo kernel. Sus partes se llaman entre sí como funciones normales, y las aplicaciones piden servicios con llamadas al sistema.

Ventajas: Alto rendimiento, porque no hay cambios de contexto ni paso de mensajes entre componentes internos.

Desventajas: Un error en cualquier driver puede causar un kernel panic y tumbar todo el sistema. Además, el kernel es muy grande y difícil de mantener.

2.2 Kernel con diseño modular: FreeBSD

Origen: Descendiente de BSD UNIX (Universidad de California, Berkeley), con primera versión en 1993. Hoy se usa en servidores y en redes.

Organización del kernel: Tiene un núcleo base y los controladores como módulos (.ko) en /boot/kernel, que se cargan y descargan con kldload y kldunload mientras el sistema funciona. Con kldstat se ven los módulos cargados. Linux hace lo mismo con los módulos LKM (lsmod, modprobe).

Ventajas: Se agrega soporte de hardware sin recompilar el kernel ni reiniciar, y el kernel base queda más pequeño.

Desventajas: Los módulos se ejecutan con los mismos privilegios que el kernel, así que un módulo defectuoso sigue siendo un riesgo.

2.3 Sistema estructurado en capas: THE

Origen: Diseñado por Edsger Dijkstra en la Technische Hogeschool Eindhoven (Países Bajos) en 1968.

Organización del kernel: Está dividido en capas numeradas y cada una solo usa los servicios de la capa inferior: capa 0 (planificación del procesador y multiprogramación), capa 1 (gestión de memoria), capa 2 (comunicación con la consola del operador), capa 3 (entrada/salida), capa 4 (programas de usuario) y capa 5 (el operador).

Ventajas: Es fácil de construir, depurar y verificar capa por capa.

Desventajas: Es difícil decidir qué va en cada capa, y atravesar varias capas para cada petición reduce el rendimiento.

2.4 Sistema con micronúcleo: MINIX 3

Origen: Creado por Andrew Tanenbaum en la Vrije Universiteit de Ámsterdam. MINIX 3 es la versión orientada a la confiabilidad.

Organización del kernel: El micronúcleo hace solo lo mínimo: planificación, comunicación entre procesos mediante mensajes (IPC) y manejo básico de interrupciones. El sistema de archivos, los drivers y la gestión de procesos corren como procesos separados en modo usuario. Si un driver falla, un servidor de reencarnación lo reinicia sin afectar al resto.

Ventajas: Es más robusto y seguro, porque las fallas quedan aisladas fuera del kernel.

Desventajas: El paso de mensajes entre componentes cuesta rendimiento frente a un kernel monolítico.

2.5 Máquina virtual: IBM VM/370

Origen: Lanzado por IBM en 1972 para los mainframes System/370. Su descendiente actual es z/VM.

Organización del kernel: Un programa de control (CP, el monitor de máquinas virtuales) está sobre el hardware real y crea varias máquinas virtuales que replican el hardware completo. Cada usuario ejecuta encima su propio sistema operativo, normalmente CMS, que es monousuario e interactivo.

Ventajas: Aísla totalmente a los usuarios y permite correr varios sistemas operativos a la vez en el mismo equipo.

Desventajas: Agrega una capa extra que consume rendimiento, y virtualizar el hardware completo es complejo.

3. Versión 6: Instalación y Clasificación

3.1 Instalar un nuevo kernel en Ubuntu (Repositorios)

```
uname -r
apt search linux-image-generic-hwe
sudo apt update
sudo apt install linux-generic-hwe-$(lsb_release -rs)
sudo reboot
```

3.2 Clasificación de SO

- BeOS, Mac OS X, newOS: Kernel Híbrido.
- Mach, MINIX: Micronúcleo.
- AdeOS, EROS, KeyKOS: Nanonúcleo.
- Opal, Mungi, BriX: SASOS (Single Address Space Operating System).
- MIT exokernel: Exonúcleo.

4. Evidencias de Ejecución (Windows 11 / WSL2 / VirtualBox)

```
$ uname -r
6.18.33.2-microsoft-standard-WSL2

$ wget linux-6.18.54.tar.xz
2026-09-28 22:30:48 URL:https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.18.54.tar.xz
[154801608/154801608]
-rw-r--r-- 1 sergio_gomez sergio_gomez 148M Sep 25 09:47 linux-6.18.54.tar.xz
```

```
#
# configuration written to .config
#
```

```
real    30m50.256s
user    293m32.196s
sys     54m55.183s
```

```
$ make kernelrelease
6.18.54-microsoft-standard-WSL2

$ ls -lh arch/x86/boot/bzImage
-rw-r--r-- 1 sergio_gomez sergio_gomez 15M Sep 28 23:01 arch/x86/boot/bzImage

$ file arch/x86/boot/bzImage
arch/x86/boot/bzImage: Linux kernel x86 boot executable, bzImage, version 6.18.54-microsoft-
standard-WSL2 (sergio_gomez@DESKTOP-8T2JB98) #1 SMP PREEMPT_DYNAMIC Mon Sep 28 22:59:50
-05 2026

$ find . -name "*.ko" | wc -l
960

$ make modules_install INSTALL_MOD_PATH=~/.kernel/rootfs
rc=0
6.18.54-microsoft-standard-WSL2

$ modinfo ../kernel/net/nsh/nsh.ko
license:      GPL v2
description:  NSH protocol
author:       Jiri Benc <jbenc@redhat.com>
intree:       Y
```

```
# cat /etc/os-release
NAME=FreeBSD
VERSION="15.1-RELEASE"
VERSION_ID="15.1"
ID=freebsd
# freebsd-version -u -k -r
15.1-RELEASE
15.1-RELEASE
15.1-RELEASE
# getconf LONG_BIT
64
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE releng/15.1-n283562-96841ea08dcf GENERIC
amd64
```

```
# cd /usr/src/sys/amd64/conf
# mkdir /root/kernels
# cp GENERIC /root/kernels/CLASEKERN
# ln -s /root/kernels/CLASEKERN
lrwxr-xr-x  1 root wheel 23 Sep 28 23:16 CLASEKERN -> /root/kernels/CLASEKERN
# grep ident CLASEKERN
ident          CLASEKERN
```

```
>>> Kernel(s) CLASEKERN built in 3924 seconds, ncpu: 6, make -j6
3933.05 real    17756.49 user    4551.05 sys
```

```
ERROR: The kernel at /boot/kernel was installed from packages.
A packaged kernel should never be updated using installkernel;
this will cause the package database to become out of sync with
the live system state. Either uninstall the packaged kernel,
or install this kernel to a different path using INSTKERNNAME.
```

```
# make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN
>>> Install kernel(s) CLASEKERN completed in 221 seconds, ncpu: 6
# ls -lh /boot/CLASEKERN/kernel
-r--r--r--  1 root wheel  28M Sep 29 00:21 /boot/CLASEKERN/kernel
# nextboot -k CLASEKERN
# init 6
```

```
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE CLASEKERN amd64
# uname -i
CLASEKERN
# sysctl kern.bootfile
kern.bootfile: /boot/CLASEKERN/kernel
# sysctl kern.ident
kern.ident: CLASEKERN
# kldstat | head -3
Id Refs Address          Size Name
  1    5 0xffffffff80200000 22fb980 kernel
  2    1 0xffffffff82c18000   3220 intpm.ko
```